ClearMind

Complete Feature Documentation

An adaptive AI chat interface that reduces accessibility barriers for users with ADHD and dyslexia. Each feature is backed by peer-reviewed research.

Research Question: How can AI chat interfaces adapt their behavior and appearance to reduce accessibility barriers for users with ADHD and dyslexia?

Created by: Hargun Malhotra, John Riley, Samia Saeed, Ella Lewis

UMD TRAILS AI Summer Camp 2026 | Participatory Governance Trail

Features at a Glance

#	Feature	File(s)
1	Adaptive System Prompts	prompts.py
2	Readability Post-Processing	readability.py, config.py
3	TL;DR-First Formatting & Chunking	formatter.py
4	Topic Drift Detection (Refocus)	refocus.py
5	Dyslexia-Friendly Display	index.html, config.py
6	ADHD Visual Optimizations	index.html, config.py
7	User-Controlled Mode Selection	index.html
8	Calming Visual Design	index.html
9	Tinted Reading Overlay	index.html
10	Graceful Error Handling	app.py, gemini_client.py
11	Session Management & New Chat	app.py, index.html
12	Full Adaptive Pipeline	app.py
13	Responsive Layout	index.html
14	Markdown Rendering	formatter.py

Feature 1: Adaptive System Prompts

ClearMind uses different system prompts for each mode (Dyslexia, ADHD, Combined). This changes HOW the AI thinks and responds, not just how the response looks. The AI is instructed to use simpler vocabulary, shorter sentences, answer-first structure, and TL;DR summaries depending on the user's needs.

Problem Solved

Current AI chatbots use one system prompt for everyone. Users with ADHD or dyslexia must manually re-prompt the AI to get accessible responses, placing the burden of adaptation on the user.

Research Evidence

> *"GAI use places the burden of adaptation onto 'power users' themselves, reinforcing normative expectations rather than challenging the systems necessitating the adaptations."*
>
> -- Glazko et al. (2025), CHI 2025

> *"There is a gap between technology development and user experience as it relates to disability needs."*
>
> -- Silverman et al. (2025), AFB Delphi study

> *"Most systems follow a 'one-size-fits-all' model, ignoring sensory sensitivity, cognitive processing differences (e.g., semantic comprehension delays in people with dyslexia), and behavioural characteristics of neurodiverse groups."*
>
> -- Panda et al. (2025), EMNLP 2025

> *"Across 3 case studies, the authors argue that a single supposedly 'correct' output often fails. Information needs vary by context, purpose, language community, and the user's own sense-making strategies."*
>
> -- Tang et al. (2026), FAccT 2026 'Cripping AI'

Implementation

- Dyslexia mode: 15-word max sentences, common words, no idioms, numbered lists, recap
- ADHD mode: TL;DR first, answer-first structure, bold key terms, no tangents, clear next steps
- Combined mode: merges both -- TL;DR + simple language + bold key phrases + labeled sections
- Each prompt is stored in prompts.py and selected server-side based on user's chosen mode

Source file(s): prompts.py

Feature 2: Readability Post-Processing

After the AI generates a response, ClearMind measures its reading level using Flesch-Kincaid Grade Level and Flesch Reading Ease. If the response is too complex for the user's mode, the system automatically sends it back to the AI with specific simplification instructions. This loop repeats up to 3 times until the text meets the target reading level.

Problem Solved

AI responses often use academic language, long sentences, and complex vocabulary. For people with dyslexia, this causes comprehension delays. For people with ADHD, dense text overloads working memory and causes disengagement.

Research Evidence

> *"With ADHD, it can be tough to read scientific papers because there are all these different terminologies and it is too much new information to keep in my working memory."*
>
> -- Giri et al. (2026), CHI 2026, participant P16

> *"Sometimes, I don't really choose to use it. It is survival mode, I am tired, slow, and class is starting in 20 minutes... I really need to know what that paper means."*
>
> -- Glazko et al. (2025), CHI 2025, participant A3

> *"For users with ADHD or dyslexia, AI responses should use familiar words, short sentences, clear headings, summaries, and small content blocks."*
>
> -- W3C (2021), Cognitive Accessibility Guidelines

Implementation

- Flesch-Kincaid Grade Level: target 6.0 or lower for dyslexia/combined, 8.0 for ADHD
- Flesch Reading Ease: target 70+ for dyslexia/combined, 60+ for ADHD
- Per-sentence word limits: 15 words for dyslexia/combined, 20 for ADHD
- Complex word ratio tracking (3+ syllable words)
- Up to 3 iterative simplification passes with specific failure feedback
- Skips simplification for very short responses (under 30 words) to avoid false triggers

Source file(s): readability.py, config.py

Feature 3: TL;DR-First Formatting and Chunking

For ADHD and combined modes, every AI response starts with a one-sentence TL;DR summary. The body is then broken into small visual chunks (3-4 sentences each) separated by subtle dividers. This implements progressive disclosure: the user gets the answer immediately and can read further if they want. Dyslexia mode also gets TL;DR and 3-sentence chunks.

Problem Solved

ADHD users lose focus when responses are long and unstructured. Working memory limitations mean dense text is hard to process. Users need the answer first, details second.

Research Evidence

> *"AI interfaces should reduce dependence on memory by preserving context, showing progress, providing reminders, allowing users to undo mistakes, and keeping important controls consistently available."*
>
> -- W3C (2021), Cognitive Accessibility Guidelines

> *"Participant feedback specifically favored progressive disclosure and clearer orientation."*
>
> -- Gunawardana et al. (2025), IndiaHCI

> *"[I was] overwhelmed by where to start and that overwhelm turns into confirmation of that deeply held belief that you're a piece of shit."*
>
> -- Giri et al. (2026), CHI 2026, participant P17

Implementation

- Extracts existing TL;DR from AI response if present (TL;DR:, TLDR:, In short:, Summary:)
- Auto-generates TL;DR from the first sentence if the AI didn't include one
- Chunks body into configurable sentence groups (3 for dyslexia/combined, 4 for ADHD)
- Preserves existing paragraph breaks before chunking
- Renders as styled HTML divs with subtle bottom borders between chunks

Source file(s): formatter.py, config.py

Feature 4: Topic Drift Detection (Refocus)

In ADHD and combined modes, ClearMind tracks the conversation topic using TF-IDF cosine similarity. When the user's latest message diverges significantly from their recent conversation history, the system gently asks if they want to return to their original topic or continue in the new direction. This is a suggestion, not a block -- respecting user agency.

Problem Solved

People with ADHD frequently lose track of their original question or task due to difficulties with sustained attention and working memory. They may go down tangents and forget what they were originally trying to accomplish.

Research Evidence

> *"I needed to plan my evening because I was a little overwhelmed. I needed to do homework, shower, pray, and eat. Neurodivergent people are not very good at staying on schedule with our basic human needs."*
>
> -- Giri et al. (2026), CHI 2026, participant P12

> *"The interface should avoid unnecessary interruptions and should not present excessive information at once."*
>
> -- W3C (2021), Cognitive Accessibility Guidelines

> *"Future research should directly address neurodivergent and disabled people's needs for improving accessibility of GAI tools... while deferring to their agency and autonomy."*
>
> -- Glazko et al. (2025), CHI 2025

Implementation

- TF-IDF vectorization of each user message
- Cosine similarity between latest message and a sliding window of recent messages
- Configurable drift threshold (default: 0.15 similarity)
- Minimum 3 messages before drift detection activates
- Identifies dominant topic words and includes them in the refocus suggestion
- Drift notice rendered as a styled banner above the AI response
- Only active in ADHD and combined modes

Source file(s): refocus.py, config.py

Feature 5: Dyslexia-Friendly Display

In dyslexia and combined modes, the frontend applies: OpenDyslexic font, 18px minimum font size, 200% line height, +12% character spacing, +20% word spacing, and 58-character max line width. These are CSS class overrides applied automatically when the user selects their mode.

Problem Solved

Standard chat interfaces use small fonts, tight spacing, and long lines that make text physically harder to track and decode for people with dyslexia.

Research Evidence

> *"Larger font sizes significantly improve readability, especially for people with dyslexia (ranging from 18 to 24 points). Larger character spacings (up to +7 to +14%) significantly improve readability for people with and without dyslexia."*
>
> -- Rello and Baeza-Yates (2017), Universal Access in the Information Society

> *"More consequential adaptations involved how assistance worked: several rewrite choices rather than one imposed answer, custom instructions, an 18-point font, 140% line spacing, and read-aloud support."*
>
> -- Goodman et al. (2022), ASSETS, LaMPost study

Implementation

- OpenDyslexic font loaded via @font-face (Regular, Bold, Italic, BoldItalic variants)
- Font size: 18px (meets Rello's 18-24pt recommendation)
- Line height: 2.0 (exceeds Goodman's 140% recommendation)
- Letter spacing: 0.12em (within Rello's +7-14% range)
- Word spacing: 0.2em for improved word boundary recognition
- Max line width: 58ch to reduce line-tracking difficulty
- Applied via CSS body class toggle -- no page reload needed

Source file(s): index.html, config.py

Feature 6: ADHD Visual Optimizations

ADHD mode applies slightly larger text (17px), increased line height (1.7), subtle letter spacing, and 65-character max line width. Combined with TL;DR-first formatting, bold key terms, and chunked responses, this creates a visually scannable layout that reduces cognitive load.

Problem Solved

Dense, unstructured text overloads working memory. Without visual hierarchy, ADHD users struggle to find the important information in a response.

Research Evidence

> *"For users with ADHD or dyslexia, AI responses should use familiar words, short sentences, clear headings, summaries, and small content blocks."*
>
> -- W3C (2021), Cognitive Accessibility Guidelines

> *"With ADHD, it can be tough to read scientific papers because there are all these different terminologies and it is too much new information to keep in my working memory."*
>
> -- Giri et al. (2026), CHI 2026, participant P16

Implementation

- Font size: 17px for AI messages
- Line height: 1.7 for comfortable reading
- Letter spacing: 0.02em for subtle improved tracking
- Max line width: 65ch to balance readability and information density
- Bold key terms rendered with accent color for visual anchoring
- TL;DR block styled with green left border for instant recognition

Source file(s): index.html, config.py

Feature 7: User-Controlled Mode Selection

Users choose their own mode (Dyslexia, ADHD, Combined) via pill-shaped buttons in the header. The system does not auto-detect disability. The default mode is Combined, providing maximum accessibility out of the box. Users can switch modes at any time without losing their conversation. The Standard/regular mode has been intentionally removed -- all modes provide accessibility adaptations.

Problem Solved

Auto-detecting disability is both technically unreliable and ethically problematic. Forcing accessibility features on users who don't want them can feel patronizing.

Research Evidence

> *"Future research should directly address neurodivergent and disabled people's needs... while deferring to their agency and autonomy in how and why they use GAI, acknowledging them as experts in navigating and mitigating both social and technological harms."*
>
> -- Glazko et al. (2025), CHI 2025

> *"The proposed framework contains 3 principles: expose the disability politics embedded in AI, honor disabled ways of knowing, and respect the continuing labor disabled people perform to make systems usable."*
>
> -- Tang et al. (2026), FAccT 2026 'Cripping AI'

Implementation

- Three mode buttons: Dyslexia, ADHD, Combined
- No Standard/regular mode -- every mode is accessibility-focused
- Default is Combined for maximum accessibility out of the box
- Mode switch updates CSS body class instantly (no page reload)
- Mode is sent with each API request so the backend adapts too
- Respects user agency: no auto-detection, no assumptions

Source file(s): index.html, app.py

Feature 8: Calming Visual Design

The interface uses a soft pastel color palette (pale greens, warm whites, muted earth tones) with gentle floating geometric SVG animations in the background. The design is minimalist and clean, inspired by Claude's interface, with no unnecessary visual noise. Scrollbars are hidden throughout to reduce visual clutter while maintaining scroll functionality.

Problem Solved

Visually busy or high-contrast interfaces can increase sensory overload and visual stress for users with ADHD and dyslexia.

Research Evidence

> *"Predictable and adjusted visuals can help soothe sensory overload and visual stress associated with ADHD and dyslexia. Helpful options include slow fluid-motion or looping geometric visuals, soft pastel or muted color palettes."*
>
> -- Design principle from ClearMind research basis

> *"The interface should avoid unnecessary interruptions and should not present excessive information at once."*
>
> -- W3C (2021), Cognitive Accessibility Guidelines

Implementation

- Soft pastel palette: accent green (#5b8a72), warm white (#f7f5f0), cream backgrounds
- Three floating SVG geometric shapes (circles, rounded rectangle, pentagon) with slow animation
- Animation uses ease-in-out with 18-25 second cycles -- slow enough to be calming, not distracting
- All scrollbars hidden via scrollbar-width: none and ::-webkit-scrollbar display: none
- Subtle box shadows instead of hard borders for depth
- Smooth transitions (180ms ease) on all interactive elements

Source file(s): index.html

Feature 9: Tinted Reading Overlay

In dyslexia and combined modes, a subtle warm-tinted overlay (rgba(255, 248, 220, 0.12)) is applied over the entire page. This simulates the effect of tinted reading glasses or colored overlay sheets. The overlay is transparent enough not to obscure content but provides a warm tone that reduces harsh white glare.

Problem Solved

Colored overlays have been shown to reduce visual stress and improve reading comfort for some people with dyslexia.

Research Evidence

> *"Larger character spacings (up to +7 to +14%) significantly improve readability for people with and without dyslexia."*
>
> -- Rello and Baeza-Yates (2017), Universal Access in the Information Society

> *"Accessibility can be approached not only in terms of text presentation, but also in terms of text content."*
>
> -- Rello and Baeza-Yates (2017)

Implementation

- Fixed-position overlay covering the entire viewport
- Pointer-events: none so it doesn't interfere with interaction
- Warm cream tint: rgba(255, 248, 220, 0.12)
- Smooth 0.4s fade transition when switching modes
- Only active in dyslexia and combined modes

Source file(s): index.html

Feature 10: Graceful Error Handling

ClearMind catches all API errors and displays friendly, plain-language messages instead of technical error dumps. Rate limit errors get a specific message asking the user to wait. Simplification pass failures are caught individually so the pipeline returns the best available result rather than failing entirely.

Problem Solved

API failures, rate limits, and network errors can produce confusing error messages or blank responses, causing frustration and anxiety.

Research Evidence

> *"AI interfaces should reduce dependence on memory by preserving context, showing progress, providing reminders, allowing users to undo mistakes."*
>
> -- W3C (2021), Cognitive Accessibility Guidelines

Implementation

- API 429 (rate limit) errors show: 'The API is temporarily rate-limited. Please wait a minute and try again.'
- Other API errors show a truncated, readable error message
- Simplification loop failures are caught per-pass -- returns best effort rather than crashing
- Network errors from the frontend show: 'Could not reach the server. Is it running?'
- Loading indicator shows during API calls with gentle pulsing dots animation

Source file(s): app.py, gemini_client.py

Feature 11: Session Management and New Chat

Each browser tab gets a unique session ID. Conversation history (last 10 turns) is sent with each API call for context. A 'New chat' button in the header clears the conversation display, resets the server-side history and drift detector, and generates a fresh session ID.

Problem Solved

Users may want to start fresh without losing the ability to choose their preferred mode. Conversation history should persist within a session but be clearable on demand.

Research Evidence

> *"AI interfaces should reduce dependence on memory by preserving context, showing progress, providing reminders, allowing users to undo mistakes, and keeping important controls consistently available."*
>
> -- W3C (2021), Cognitive Accessibility Guidelines

Implementation

- Unique session ID generated per page load (session-{timestamp})
- Server stores conversation history per session (last 10 turns sent for context)
- New chat button calls /api/reset to clear server-side state
- New session ID generated on reset to ensure clean state
- Drift detector reset alongside conversation history
- Welcome screen re-displayed after reset

Source file(s): app.py, index.html

Feature 12: Full Adaptive Pipeline

The Flask server orchestrates all features into a single pipeline: (1) mode-specific system prompt selection, (2) API call with conversation history, (3) readability scoring and iterative simplification, (4) topic drift detection for ADHD/combined, (5) TL;DR extraction and chunking, (6) Markdown-to-HTML conversion, (7) conversation history update. The response includes the formatted HTML, readability metrics, and drift status.

Problem Solved

Individual accessibility features are insufficient without integration. The system needs to orchestrate prompt selection, API calls, readability enforcement, drift detection, and formatting into a seamless pipeline.

Research Evidence

> *"GAI use places the burden of adaptation onto 'power users' themselves."*
>
> -- Glazko et al. (2025), CHI 2025

> *"There is a gap between technology development and user experience as it relates to disability needs."*
>
> -- Silverman et al. (2025), AFB Delphi study

Implementation

- 7-step pipeline executed on every message
- System prompt selected based on user's mode
- Conversation history (last 10 turns) provides multi-turn context
- Readability loop: score, fail, simplify, re-score (up to 3x)
- Drift detection only runs for ADHD and combined modes
- Formatter applies TL;DR and chunking based on mode's config
- Response includes HTML, readability metrics, and drift info
- All steps logged with session ID for debugging

Source file(s): app.py

Feature 13: Responsive Layout

The layout uses flexbox with a max-width container (860px) centered on desktop. Media queries at 768px and 480px adapt the header, mode selector, chat area, and input area for smaller screens. Mode buttons wrap to a full-width row on mobile. The readability badge hides on small screens to save space.

Problem Solved

Users may access ClearMind from phones, tablets, or desktops. The interface must adapt to all screen sizes without breaking accessibility features.

Research Evidence

> *"More consequential adaptations involved how assistance worked: several rewrite choices rather than one imposed answer, custom instructions."*
>
> -- Goodman et al. (2022), ASSETS, LaMPost study

Implementation

- Max-width: 860px container with auto margins for desktop centering
- 768px breakpoint: header wraps, mode selector takes full width, reduced padding
- 480px breakpoint: further reduced padding/font sizes for small phones
- Background geometric shapes fade to 15% opacity on mobile to reduce visual noise
- Message bubbles expand to 100% width on mobile
- Input area adapts with reduced padding on smaller screens

Source file(s): index.html

Feature 14: Markdown Rendering

ClearMind converts markdown in AI responses to proper HTML before sending to the frontend. This supports bold text, numbered and bulleted lists, headers, and paragraph breaks. The rendered HTML is styled consistently with the mode's visual settings.

Problem Solved

AI models often return responses with markdown formatting (bold, lists, headers). Without proper rendering, raw markdown syntax clutters the display.

Research Evidence

> *"For users with ADHD or dyslexia, AI responses should use familiar words, short sentences, clear headings, summaries, and small content blocks."*
>
> -- W3C (2021), Cognitive Accessibility Guidelines

Implementation

- Python markdown library with 'extra', 'sane_lists', and 'nl2br' extensions
- Applied to both TL;DR and chunk content before HTML output
- Bold text rendered with accent color for visual emphasis
- Lists properly indented and spaced for readability
- Headers styled at consistent sizes within message bubbles

Source file(s): formatter.py

Architecture Overview

Pipeline Flow

1. User sends a message with their selected mode (Dyslexia / ADHD / Combined)
2. Server selects the mode-specific system prompt from prompts.py
3. API call to OpenRouter with system prompt + last 10 conversation turns
4. Readability engine scores the response (Flesch-Kincaid Grade + Reading Ease)
5. If too complex: send back to API with specific simplification instructions (up to 3x)
6. If ADHD/Combined mode: run TF-IDF drift detection on conversation history
7. Formatter extracts/generates TL;DR and chunks the body into digestible blocks
8. Markdown converted to HTML; drift notice prepended if detected
9. HTML response sent to frontend with readability metrics and drift status

File Structure

File	Purpose	Lines
config.py	All constants, thresholds, display settings	~109
prompts.py	Adaptive system prompts per mode	~98
readability.py	Flesch-Kincaid engine + simplification loop	~285
refocus.py	TF-IDF drift detection for ADHD	~225
formatter.py	TL;DR extraction + chunking + HTML conversion	~208
gemini_client.py	OpenRouter API wrapper	~105
app.py	Flask server + full pipeline orchestration	~250
index.html	Frontend: chat UI, mode switching, CSS	~580
requirements.txt	Python dependencies	4

Setup Instructions

1. Install dependencies: `pip install -r requirements.txt`
2. Create a `.env` file with your API key: `OPENROUTER_API_KEY=sk-or-...`
3. Run the server: `py app.py`
4. Open `http://localhost:5000` in your browser